

Operating Systems

No. 5

Sarun Intakosum

Interprocess Communication (IPC)

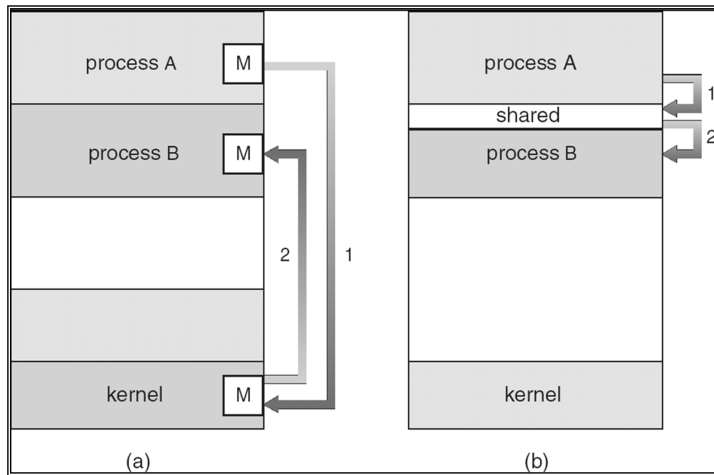
Interprocess Communication

- Processes within a system may be independent or cooperating
- Cooperative processes can influence or be influenced by other processes, including data exchange.
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
- Cooperating processes need **interprocess communication (IPC)**
- Three models of IPC
 - Signal
 - Shared memory
 - Message passing

Signals

- In an operating system, a signal is a way for one program (or even the OS itself) to interrupt another program.
- Examples:
 - Errors or Exceptions: "Hey, something went wrong here!" (e.g., trying to divide by zero)
 - External Events: "The user just pressed Ctrl+C!" (requesting the program to stop)
 - Notifications: "Child process finished!" (a message from a child to its parent)
 - Alarms: "Time's up!" (after a set period)
- So signal is a software interrupts that notify a process that an event has occurred
- Signals carry minimal information (typically just the signal number), so they are often used for simple notifications.
- Processes may catch, use OS default, or mask a signal
 - To catch a signal, one must define a specific routine that the operating system invokes when it provides the signal.
 - Use operating system's default action to handle the signal
 - Masking a signal involves directing the operating system to withhold signals of a specific type until the process removes the signal mask.

Communications Models



UNIX Interprocess Communications

- Signal
- Pipe
- Shared memory

Signal

- `signal.h` is a header file in the C Standard Library, specifically designed for handling signals in Unix-like operating systems.
- It provides definitions and declarations necessary for signal handling, allowing programs to interact with and respond to signals.
- All signal names (that represent signal number) start with `SIG`.

Signal (Cont.)

- `SIGFPE` An arithmetic exception (e.g., division by zero, overflow and underflow) has occurred.
- `SIGINT` An interrupt (e.g., Ctrl-C) has occurred.
- `SIGTERM` Software termination.
- `SIGQUIT` (non-ANSI C) Quit execution (e.g., Ctrl-\\).
- `SIGUSR1` and `SIGUSR2` (e.g., non-ANSI C) User-defined signal 1 and 2.
- `SIGKILL` (non-ANSI C) Kill the process (cannot be caught or ignored)

Handling Signals: Function `signal()`

- The most important function for handling signals is `signal()`.
- `signal()`:
 - Sets a function as the handler for a specific signal.

`signal()` (Contd.)

- `signal(int signum, void (*handler)(int))`
- `signal()` requires two arguments:
 - The first one is an integer that is a signal number. In practice, we always pass the signal names discussed earlier.
 - The second one is the pointer to a signal handler function. For simplicity, you can think of it as the function's name
 - However, the signal handler function must be a function that:
 - Accepts an integer argument and returns nothing. The integer argument is a signal number.
- In summary, the `signal()` function tells the OS that if it detects the signal specified in the first argument, the signal handler specified in the second argument is called.
- `sig_demo.c`

Handling Multiple Signal Types

- There are two choices:
 - Install a signal handler for each signal.

```
signal(SIGINT, int_handler);
signal(SIGQUIT, quit_handler);
...
void int_handler(int sig)
{
    .....
}
void quit_handler(int sig)
{
    .....
}
```

Handling Multiple Signal Types (Contd.)

- Create a single handler whose only input is the signal to be sent:
`signal(SIGINT, sig_handler);`
`signal(SIGALRM, sig_handler);`

```
void sig_handler(int sig)
{
    .....
    switch (sig) {
        case SIGINT : .....
        case SIGALRM : ....
    }
}
```

`multiple_sig.c`

Sending a Signal: raise() function

- raise() function is used to send a signal to the executing process itself.
- The following is the function prototype:
 - `int raise(int sig)`
 - Where `sig` is a signal number to be sent to the current process. This can be any valid signal number defined in `signal.h`, such as `SIGINT`, `SIGTERM`, etc.
 - The `raise()` function returns 0 on success.
 - If an error occurs, it returns a nonzero value.
- When `raise(sig)` is called:
 - The signal specified by `sig` is sent to the calling process.
 - The process's signal handler for that signal (if one is set) is invoked.
 - If no custom handler is set, the default action associated with the signal is performed.
 - This could be terminating the process, ignoring the signal, or stopping/continuing execution.
- To send a signal to other processes, the system call `kill()` should be used.
- `raise.c`

Sending a Signal to Another Process: System Call kill()

- The `kill()` function in C is used to send a signal to a specified process or a group of processes.
- This function is typically used for inter-process communication, allowing one process to notify another process of certain events or to control its execution (e.g., terminate it).
- The following is the prototype of `kill()`:
 - `int kill(pid_t pid, int sig)`
 - `pid =>` The process ID or group of processes to which the signal will be sent.
 - Positive value: Sends the signal to the process with the specified PID.
 - 0: Sends the signal to all processes in the same process group as the calling process.
 - -1: Sends the signal to all processes for which the calling process has permission to send signals, except for process 1 (init).
 - Negative value (other than -1): Sends the signal to all processes whose process group ID is equal to the absolute value of the `pid`.
 - `sig =>` The signal number to be sent.
 - The `kill()` function returns 0 on success.
 - If an error occurs, it returns -1
- `kill_demo.c`

kill command

- The `kill` command can be used to send any signal to a process.
- The `kill` command has the following form:
 - `kill -signal pid pid pid ... pid`
- The first argument, shown in boldface, is the signal name that represents the signal to be sent.
- This signal name does not have to include `SIG`. For example, if you want to send signals `SIGINT` you can use `INT`.
- You can also use signal number
- One signal number that you should know is 9 which is `SIGKILL`.
- Use `kill -9 pid` to kill a process that is so stubborn.
- `SIGKILL` is a signal that the process cannot handle or ignore; therefore, the process that got `SIGKILL` must be terminated.
- If the signal name is missing, the default `SIGTERM` is used.

Message Passing

- Message-based interprocess communication
- IPC facility provides two operations:
 - **send(message)** – message size fixed or variable
 - **receive(message)**
- If *P* and *Q* wish to communicate, they need to:
 - establish a *communication link* between them
 - exchange messages via `send/receive`
- Messages can be passed in one direction at a time
 - One process is the sender and the other is the receiver
- Message passing can be bidirectional
 - Each process can act as either a sender or a receiver
- Popular implementation is a pipe
 - A region of memory protected by the OS that serves as a buffer, allowing two or more processes to exchange data

Synchronization

- Message passing may be either blocking or non-blocking
- **Blocking** is considered **synchronous**
 - **Blocking send** has the sender block until the message is received
 - **Blocking receive** has the receiver block until a message is available
- **Non-blocking** is considered **asynchronous**
 - **Non-blocking send** has the sender send the message and continue
 - **Non-blocking receive** has the receiver receive a valid message or null

Pipes

- An anonymous pipe (we will call pipe for short) is used for one-way communication of a stream of bytes.
- The command to create a pipe is `pipe()`, which takes an array of two integers. It fills in the array with two file descriptors that can be used for low-level I/O.

Creating a Pipe

- `int pfd[2];`
- `pipe(pfd);`



`pfd[1]`

`pfd[0]`

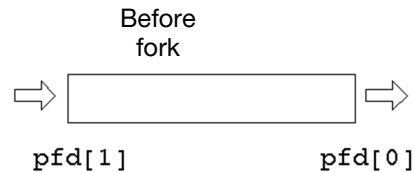
write to
this end

read from
this end

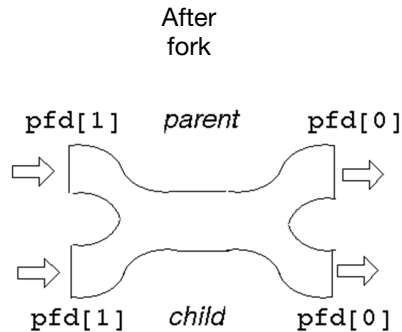
I/O with a pipe

- These two file descriptors can be used for block I/O
 - `write(pfd[1], buf, size);`
 - `read(pfd[0], buf, SIZE);`

Fork and a pipe ()

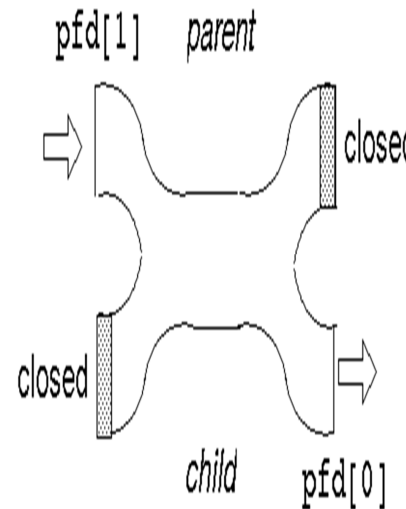


This gives two read ends and two write ends. Either process can write into the pipe, and either can read from it.



https://wiki.control.fel.cvut.cz/pos/os_api/unix-8.html

Fork and a pipe (Contd.)

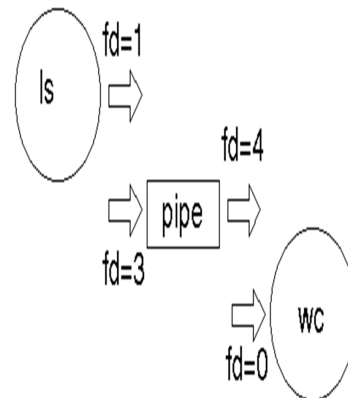


- To ensure consistent and expected behavior, one of the processes must close its read end, while the other process must close its write end.
- Subsequently, it will revert to being a straightforward pipeline.
- Let's say the parent is the sender and the child is the receiver.
- The parent closes its read end.
- The child closed its write end.
- pipe.c

https://wiki.control.fel.cvut.cz/pos/os_api/unix-8.html

Example of pipe

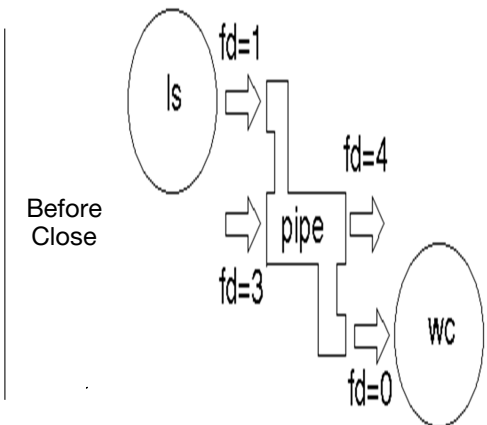
- To implement `"ls | wc"` the shell will have created a pipe and then forked.
- The parent will exec to be replaced by `"ls"`, and the child will exec to be replaced by `"wc"`
- The write end of the pipe may be descriptor 3 and the read end may be descriptor 4.
- `"ls"` normally writes to 1 and `"wc"` normally reads from 0.
- How do these get matched up?



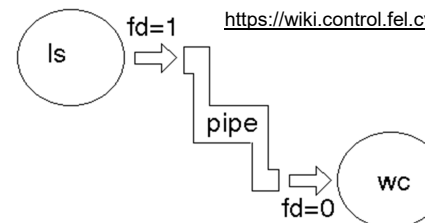
https://wiki.control.fel.cvut.cz/pos/os_api/unix-8.html

dup2 function

- The `"dup2"` function takes an existing file descriptor, and another one that it "would like to be".
- Here, `fd=3` would also like to be 1, and `fd=4` would like to be 0.
- So we `dup2 fd=3 as 1`, and `dup2 fd=4 as 0`.
- Then the old `fd=3` and `fd=4` can be closed as they are no longer needed.
- `dup2()` is useful for redirecting file descriptors, such as redirecting standard output (stdout) to a file.



After close



https://wiki.control.fel.cvut.cz/pos/os_api/unix-8.html

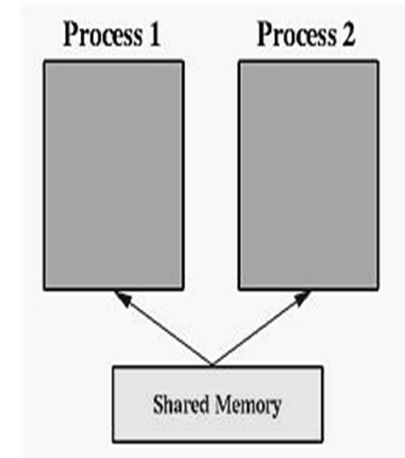
dup2.c

read/write from/to Pipe

- A process may want to both write to and read from a child.
- In this case it creates two pipes.
- One of these is used by the parent for writing and by the child for reading.
- The other is used by the child for writing and the parent for reading.
- `twopipe.c`

What is Shared Memory?

- The figure shows two processes and their address spaces.
- The yellow rectangle is a shared memory attached to both address spaces and both process 1 and process 2 can have access to this shared memory as if the shared memory is part of its own address space.
- Note that, race conditions may occur if memory accesses are not handled properly.



<https://www.csl.mtu.edu/cs4411.ck/www/NOTES/process/shm/what-is-shm.html>

What is Shared Memory? (Contd.)

- Shared memory is a feature supported by UNIX-like operating systems.
- One process must explicitly ask for an area, using a key, to be shared by other processes. This process will be called the server.
- All other processes, the clients, that know the shared area can access it.
- However, there is no protection to a shared memory and any process that knows it can access it freely.
- To protect a shared memory from being accessed at the same time by several processes, a synchronization protocol must be setup.

A general scheme of using shared memory (Server)

- For a server, it should be started before any client. The server should perform the following tasks:
 - Ask for a shared memory with a memory key and memorize the returned shared memory ID. This is performed by system call `shmget()`.
 - Attach this shared memory to the server's address space with system call `shmat()`.
 - Initialize the shared memory, if necessary.
 - Do something and wait for all clients' completion.
 - Detach the shared memory with system call `shmdt()`.
 - Remove the shared memory with system call `shmctl()`.

A general scheme of using shared memory (client)

- For the client part, the procedure is almost the same:
 - Ask for a shared memory with the same memory key and memorize the returned shared memory ID.
 - Attach this shared memory to the client's address space.
 - Use the memory.
 - Detach all shared memory segments, if necessary.
 - Exit.

Programming Shared Memory

- A shared memory segment is identified by a unique integer, the shared memory ID.
- The program should start with the following lines:
 - `#include <sys/types.h>`
 - `#include <sys/ipc.h>`
 - `#include <sys/shm.h>`

Key

- Unix requires a key of type `key_t` defined in file `sys/types.h` for requesting resources such as shared memory segments, message queues and semaphores.
- A key is simply an integer of type `key_t`; however, you should not use `int` or `long`, since the length of a key is system dependent.
- There are three different ways of using keys, namely:
 - a specific integer value (e.g., 123456)
 - a key generated with function `ftok()`
 - a uniquely generated key using `IPC_PRIVATE` (i.e., a private key).

Key (Contd.)

- The first way is the easiest one; however, its use may be very risky since a process can access your resource as long as it uses the same key value to request that resource.
- The following example assigns 1234 to a key:
 - `key_t SomeKey;`
 - `SomeKey = 1234;`

ftok()

- The ftok() function has the following prototype:
- `key_t ftok( const char *path, /* a path string */`
- `int id /* an integer value */);`
- Function ftok() takes a character string that identifies a path and an integer (usually a character) value, and generates an integer of type key_t based on the first argument with the value of id in the most significant position.
- Thus, as long as processes use the same arguments to call ftok(), the returned key value will always be the same.
- The most commonly used value for the first argument is ".", the current directory.

ftok() (Contd.)

- If all related processes are stored in the same directory, the following call to ftok() will generate the same key value:
 - `#include <sys/types.h>`
 - `#include <sys/ipc.h>`
 - `key_t SomeKey;`
 - `SomeKey = ftok(".", 'x');`

IPC_PRIVATE

- In this case, the system will generate a unique key and guarantee that no other process will have the same key.
- If a resource is requested with IPC_PRIVATE in a place where a key is required, that process will receive a unique key for that resource.
- Since that resource is identified with a unique key unknown to the outsiders, other processes will not be able to share that resource and, as a result, the requesting process is guaranteed that it owns and accesses that resource exclusively.
- IPC_PRIVATE is normally shared between related process such as parent/child, child/child.

Asking for a Shared Memory Segment - shmget()

- It is defined as follows:
- `shm_id = shmget( key_t k, /* the key for the segment */`
- `int size, /* the size of the segment */`
- `int flag); /* create/use flag */`
- In the above definition, k is of type key_t or IPC_PRIVATE. size is the size of the requested shared memory.
- The purpose of flag is to specify the way that the shared memory will be used. For our purpose, only the following two values are important:
- IPC_CREAT | 0666 for a server (i.e., creating and granting read and write access to the server)
- 0666 for any client (i.e., granting read and write access to the client)
- If shmget() can successfully get the shared memory, its function value is a non-negative integer, the shared memory ID; otherwise, the function value is negative.

Example of using shmget()

- `#include <sys/types.h>`
- `#include <sys/ipc.h>`
- `#include <sys/shm.h>`
- `#include <stdio.h>`
- `int shm_id; /* shared memory ID */`
- `shm_id = shmget(IPC_PRIVATE, 4*sizeof(int), IPC_CREAT | 0666);`
- `if (shm_id < 0) {`
- `printf("shmget error\n");`
- `exit(1);`
- `}`
- If a client wants to use a shared memory created with `IPC_PRIVATE`, it must be a child process of the server, created after the parent has obtained the shared memory, so that the private key value can be passed to the child when it is created.

Attaching a Shared Memory Segment to an Address Space - shmat()

- After a shared memory ID is returned, the next step is to attach it to the address space of a process.
- This is done with system call `shmat()`. The use of `shmat()` is as follows:
- `shm_ptr = shmat(int shmid, /* shared memory ID */`
- `const void *shmaddr, /* attached address, always NULL*/`
- `int flag); /* access flag, always 0 */`

shmat() (Contd.)

- System call `shmat()` accepts a shared memory ID, `shm_id`, and attaches the indicated shared memory to the program's address space.
- The returned value is a pointer of type `(void *)` to the attached shared memory. Thus, casting is usually necessary.
- If this call is unsuccessful, the return value is `-1`.
- Normally, the second parameter is `NULL`. Let the OS desire the attached memory address.
- If the flag is `SHM_RDONLY`, this shared memory is attached as a read-only memory; Typically, this is set to `0` for default behavior.

Detaching a Shared Memory Segment - shmdt()

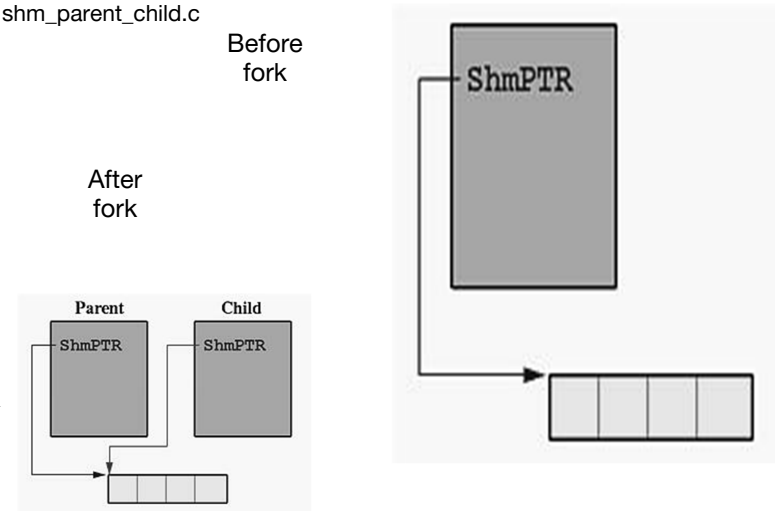
- System call `shmdt()` is used to detach a shared memory.
- After a shared memory is detached, it cannot be used.
- However, it is still there and can be re-attached back to a process's address space, perhaps at a different address.
- To remove a shared memory, use `shmctl()`, discuss next.
- The only argument of the call to `shmdt()` is the shared memory address returned by `shmat()`. Thus, the following code detaches the shared memory from a program:
 - `shmdt(shm_ptr);`
- where `shm_ptr` is the pointer to the shared memory.
- This pointer is returned by `shmat()` when the shared memory is attached.
- If the detach operation fails, the returned function value is non-zero.

Removing a Shared Memory Segment – shmctl()

- To remove a shared memory segment, use the following code:
- `shmctl(shm_id, IPC_RMID, NULL);`
- where `shm_id` is the shared memory ID. `IPC_RMID` indicates this is a remove operation.
- Note that after the removal of a shared memory segment, if you want to use it again, you use `shmget()` followed by `shmat()`.

Parent and Child Share Memory

- see `shm_parent_child.c`



Client-Server Shared Memory

- see `server.c`, `client.c`
- Note that the above code assumes the server and client programs are in the current directory.
- For the client to run correctly, the server must be started first and the client can only be started after the server has successfully obtained the shared memory.